Lucrator

Lucrator Integration Steps

The complete, click-by-click guide to deploying the lead saver + chatbot (booking & buying agent) and onboarding each client.

Two parts: **Part 1** is the one-time setup (done once, ever). **Part 2** is what you repeat for every new client — kept deliberately short.

Generated June 30, 2026 · Lucrator / TycheAS

What you are setting up

Each client gets two things: a **lead saver** (a dashboard backed by a Google Sheet) and a **chatbot** that answers visitors, books appointments, and records sales straight into that Sheet. Bookings show on the dashboard calendar.

The system is built so the hard work happens **once**. After that, adding a client is a short, repeatable checklist with **no re-deploying of code or scripts**.

The moving parts

- **Platform Apps Script** — one Google Apps Script, deployed once. It is the database engine for ALL clients: it creates each client's private Google Sheet and routes data to it.
- **Backend** — one server (Render), deployed once. It runs the chatbot brain and writes bookings/sales to the right client's Sheet.
- **Client dashboard** — the lead saver web app, deployed per client (just config values, no code edits).
- **Chatbot channels** — the widget on the client's website, and/or their Instagram & Facebook DMs.

> **Plain-English summary**
> Part 1 = build the factory once. Part 2 = press a few buttons to roll out each new client. You will not touch the Apps Script or backend code again when adding clients.

Before you start, make sure you have

- A **Google account** (for the Apps Script + the client Sheets).
- A **GitHub account** with this repo (costhaven-byte/TycheAS).
- A **Render account** (free) — or any Node host — for the backend.
- A **chatbot/LLM API key** (this guide uses Google Gemini; any OpenAI-compatible key works).
- For the DM channel only: a **Meta (Facebook) developer app** and the client's Page + Instagram.

PART 1 — One-time setup (do this once, ever)

Step 1A — Get a chatbot (LLM) API key

1. Open `https://aistudio.google.com/apikey` in your browser.
2. Sign in with your Google account.
3. Click **Create API key** (create or pick a project when asked).
4. Copy the key it shows. Keep it somewhere safe for Step 1C.

> **Heads up — model/quota**
> On the free tier the model `gemini-2.0-flash` may have a quota of 0. Use `gemini-2.5-flash` (it has free quota), or enable billing for higher limits.

Step 1B — Deploy the Platform (the database for ALL clients)

This is the one Apps Script that creates and serves every client's Google Sheet.

1. Open `https://script.google.com` and click **New project**.
2. In the editor, delete the default `function myFunction(){}` code.
3. Open the file **google/Platform.gs** from this repo, copy ALL of it, and paste it into the editor.
4. Click the disk icon (**Save project**). Name it e.g. “Lucrator Platform”.
5. At the top, in the function dropdown choose **setup**, then click **Run**.
6. A permissions popup appears → click **Review permissions** → choose your Google account → **Advanced** → “Go to (project) (unsafe)” → **Allow**. (This is your own script; that is normal.)
7. Open **Execution log** (bottom panel). Copy the two values it prints: the **ADMIN token** (starts with `admin_`) and the master registry sheet URL. Save the ADMIN token for Step 1C.
8. Top-right: **Deploy** → **New deployment**.
9. Click the gear next to “Select type” → choose **Web app**.
10. Set **Execute as: Me** and **Who has access: Anyone**.
11. Click **Deploy**, authorize again if asked, then copy the **Web app URL** (it ends in `/exec`). Save it — this is your APPS_SCRIPT_URL.

> **You now have two values you will reuse forever**
> **APPS_SCRIPT_URL** (the /exec link) and the **ADMIN token**. You only revisit this script if you change Platform.gs itself (then: Deploy → Manage deployments → Edit → New version).

Step 1C — Deploy the backend (the chatbot brain)

The repo ships a `render.yaml` so Render can deploy it directly.

1. Make sure the code is pushed to GitHub (this guide is committed with it).
2. Go to `https://render.com`, sign in, and connect your GitHub.
3. Click **New +** → **Blueprint**, pick the **TycheAS** repo. Render reads render.yaml and proposes the service.
4. Click **Apply** / **Create** to start the first deploy.
5. Open the service → **Environment** tab → add the variables in the table below → **Save** (this triggers a redeploy).
6. When it finishes, copy the service's public URL (e.g. `https://lucrator-backend.onrender.com`). Save it.

Environment variables to set (see `backend/.env.example` for the full list):

Setting	Value
`CHATBOT_API_KEY`	Your Gemini key from Step 1A
`CHATBOT_BASE_URL`	https://generativelanguage.googleapis.com/v1beta/openai
`CHATBOT_MODEL`	gemini-2.5-flash
`APPS_SCRIPT_URL`	The /exec URL from Step 1B
`CRM_API_TOKEN`	The ADMIN token from Step 1B
`API_KEY`	Any long random string (protects the backend's own endpoints)
`FRONTEND_URL / PRODUCTION_FRONTEND_URL`	Your dashboard/site origins (for CORS)
`META_APP_ID / META_APP_SECRET`	From your Meta app (only if using DMs)
`WEBHOOK_VERIFY_TOKEN`	Any random string you invent (only if using DMs)

> **Local testing instead of Render?**
> You can run the backend locally with `cd backend && npm install && npm start` and put the same values in `backend/.env`. But for live website/DM use it must be on a public HTTPS URL.

Step 1D — Register the Meta webhook (ONLY if using Instagram/Facebook DMs)

Skip this if your chatbot will only live on websites.

1. Go to `https://developers.facebook.com` → your app.
2. In the left menu open **Webhooks** (or Messenger/Instagram → Configuration).
3. Set **Callback URL** = `<backend URL>/api/meta/webhook`.
4. Set **Verify token** = the same value as WEBHOOK_VERIFY_TOKEN from Step 1C.
5. Click **Verify and save** (the backend answers the handshake automatically).

6 Subscribe to the fields: **messages** (and **feed** / **comments** if you want comment replies).

> **App Review (for real, non-test clients)**
> While your Meta app is in Development mode, only accounts with a role on the app (admin/developer/tester) can be connected — fine for demos. To onboard real clients you must pass **App Review** for the messaging permissions and have your business verified.

PART 2 — Add a client (repeat per client)

Each new client is the four short steps below. None of them require redeploying the Platform or backend.

Step 2A — Create the client's Google Sheet (one call)

This auto-creates their private Sheet with **Leads**, **Bookings**, and **Sales** tabs.

1. Take the APPS_SCRIPT_URL and ADMIN token from Step 1B.
2. In a terminal (or browser address bar) run the call below, replacing the placeholders and the client name (spaces become %20):

```
curl -L "&lt;APPS_SCRIPT_URL&gt;?token=&lt;ADMIN_TOKEN&gt;&amp;action=provision&amp;clien
```

You get back JSON like this — **save all three values**:

```
{ "ok": true, "client": {
    "clientId": "C-260630-...",
    "spreadsheetId": "1AbC...",
    "url": "https://docs.google.com/spreadsheets/d/1AbC.../edit",
    "token": "cli_..."
}}
```

- **url** — the client's live database. Share it with them (or keep it).
- **clientId** — used in Steps 2B and 2C.
- **token** — the client's own token; used in Step 2B (dashboard).

Step 2B — Deploy the client's dashboard (the lead saver)

1. Create the client's dashboard build (from the lead-saving-system template).
2. Set its config to point at the shared Platform: APPS_SCRIPT_URL (from 1B), the client's **clientId** and **token** (from 2A).
3. Set the brand name / accent color / login password for this client (cosmetic).
4. Deploy it (e.g. Vercel) and give the client their dashboard link + password.

> **The booking calendar**
> The dashboard's calendar reads the client's **Bookings** tab, so every appointment the chatbot makes appears on their calendar automatically — nothing extra to wire per client.

Step 2C — Put the chatbot on the client's website

1. Embed the chatbot widget on the client's site.
2. Set `VITE_CHATBOT_CLIENT_ID` = the client's **clientId** (from 2A) and `VITE_API_BASE_URL` = your backend URL (from 1C).
3. That ties every booking/sale from this widget to the correct client's Sheet.

Step 2D — Connect the client's Instagram & Facebook DMs (optional)

1. Ask the client to make their **Instagram a Professional account** and have a **Facebook Page**.

2. Have them **link the Instagram account to the Facebook Page** (Page Settings → Linked accounts). This step is essential — DMs fail without it.
3. In Instagram: Settings → Messages → **Connected tools** → **allow access to messages**.
4. The client adds your business as a **Partner** and shares their Page + Instagram with you (Business Settings → Partners), OR you add the assets to your own business.
5. In your Business Settings → **System users** → **lucrator backend**, assign the client's Page + Instagram as assets, then **Generate token**.
6. On the backend (Render → Environment), set `FACEBOOK_PAGE_ID`, `INSTAGRAM_ACCOUNT_ID`, and `META_USER_ACCESS_TOKEN` to this client's values, and `CRM_CLIENT_ID` to their clientId. Save (redeploys).
7. Send a test DM to the client's Instagram — the agent should reply and any booking lands in their Sheet.

> **Many clients on DMs at once**
> One backend currently serves one DM client at a time via CRM_CLIENT_ID. Serving many clients' DMs from a single backend needs a Page-ID → clientId map (a small addition). Website widgets are already fully multi-client via VITE_CHATBOT_CLIENT_ID.

Per-client cheat sheet (the short version)

Once Part 1 is done, this is the whole loop for each new client:

1. **Provision:** run the curl call → copy clientId, token, sheet URL.
2. **Dashboard:** deploy it with APPS_SCRIPT_URL + clientId + token + brand. Hand the client the link.
3. **Website widget:** embed with VITE_CHATBOT_CLIENT_ID = clientId.
4. **(Optional) DMs:** share + assign Meta assets, set the client's Meta IDs + CRM_CLIENT_ID on the backend.

Security checklist (before real launch)

- **Rotate** the demo Gemini key and the Meta App Secret (both were shared in chat during setup).
- Never put the **ADMIN token** in any browser/frontend — backend only. Client dashboards use their own per-client token.
- Each per-client token can only read/write that client's own Sheet.
- Free Gemini tier has low rate limits — enable billing for production traffic.
- Never commit real client tokens/passwords to a public repo.

Troubleshooting

- **Chatbot says it can't book:** APPS_SCRIPT_URL, CRM_API_TOKEN, or CRM_CLIENT_ID is missing on the backend — it stays in answer-only mode until all three are set.
- **429 / quota error from Gemini:** switch CHATBOT_MODEL to gemini-2.5-flash, or enable billing.
- **Instagram DM not answered:** confirm the IG is linked to the Page, message access is on, the webhook is verified, and the app is subscribed to the Page.
- **“Unknown clientId”:** the clientId doesn't match a provisioned client — re-check the value from Step 2A.
- **Provision call fails:** make sure you used the ADMIN token (not a client token) and the correct /exec URL.